

from google.colab import drive
drive.mount('/content/drive')

import pandas as pd

Load the dataset from your Drive
file_path = '/content/drive/MyDrive/Papers/Paper 4/DAY-3/cellage3.tsv'

Read the .tsv file
df = pd.read_csv(file_path, sep='\t')

Preview first few rows
print("Shape:", df.shape)
df.head()

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Shape: (949, 7)

	Entrez ID	Gene symbol	Gene name	Cancer	Cell	Type of senescence	Senescence Effect	Reference
0	22848	AAK1	AP2 associated kinase 1		No	Unclear	Induces	26583757
1	5243	ABCB1	ATP binding cassette subfamily B member 1		Yes	Stress-induced	Induces	10825123
2	368	ABCC6	ATP binding cassette subfamily C member 6		Yes	Unclear	Inhibits	28536638
3	51225	ABI3	ABI family member 3		Yes	Oncogene-induced	Induces	21223585
4	25890	ABI3BP	ABI family member 3 binding protein		Yes	Unclear	Induces	18559958

Save cleaned version as CSV
df.to_csv('/content/drive/MyDrive/Papers/Paper 4/DAY-3/cellage3.csv', index=False)

print("Saved cellage3.csv successfully!")

Saved cellage3.csv successfully!

!pip install SPARQLWrapper pandas

Collecting SPARQLWrapper
 Downloading SPARQLWrapper-2.0.0-py3-none-any.whl.metadata (2.0 kB)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Collecting rdflib<=6.1.1 (from SPARQLWrapper)
 Downloading rdflib-7.2.1-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: numpy<=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil<=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz<=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata<=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: six<=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil<=2.8.2->pandas) (1.17.0)
Requirement already satisfied: pyparsing<4,>=2.1.0 in /usr/local/lib/python3.12/dist-packages (from rdflib<=6.1.1->SPARQLWrapper) (3.2.5)
Downloading SPARQLWrapper-2.0.0-py3-none-any.whl (28 kB)
Downloading rdflib-7.2.1-py3-none-any.whl (565 kB)
565.4/565.4 kB 27.5 MB/s eta 0:00:00

Installing collected packages: rdflib, SPARQLWrapper
Successfully installed SPARQLWrapper-2.0.0 rdflib-7.2.1

from SPARQLWrapper import SPARQLWrapper, JSON
import pandas as pd

Connect to SynBioHub public SPARQL endpoint
endpoint_url = "https://synbiohub.org/sparql"

Example query: fetch iGEM parts with promoters, genes, or circuits in their description
query = """
PREFIX sbol: <http://sbols.org/v2#>
PREFIX dcterms: <http://purl.org/dc/terms/>
SELECT ?part ?name ?description
WHERE {
 ?part a sbol:ComponentDefinition .
 ?part dcterms:title ?name .
 OPTIONAL { ?part dcterms:description ?description . }
 FILTER(CONTAINS(LCASE(?description), "promoter") ||
 CONTAINS(LCASE(?description), "gene") ||
 CONTAINS(LCASE(?description), "circuit"))
}
LIMIT 200
"""

sparql = SPARQLWrapper(endpoint_url)
sparql.setQuery(query)
sparql.setReturnFormat(JSON)
results = sparql.query().convert()

records = []
for r in results["results"]["bindings"]:
 records.append({

```

records = []
for r in results["results"]["bindings"]:
    records.append({
        "Part URI": r["part"]["value"],
        "Name": r.get("name", {}).get("value", ""),
        "Description": r.get("description", {}).get("value", "")
    })

igem_df = pd.DataFrame(records)
igem_df.head()

```

	Part URI	Name	Description
0	https://synbiohub.org/public/igem/BBa_K1510929/1	BBa_K1510929	Circuit injected in Streptococcus mutans that ...
1	https://synbiohub.org/public/igem/BBa_K1367003/1	BBa_K1367003	This plasmid containing siRNA for LAR gene in ...
2	https://synbiohub.org/public/igem/BBa_K1367004/1	BBa_K1367004	siRNA for LAR (leucoanthocyanidin reductase) ...
3	https://synbiohub.org/public/igem/BBa_K1420006/1	BBa_K1420006	phsABC Generator
4	https://synbiohub.org/public/igem/BBa_K1336007/1	BBa_K1336007	LacI Expression Cassette with BsDyP gene

Next steps: [Generate code with igem_df](#) [New interactive sheet](#)

```

igem_df.to_csv('/content/drive/MyDrive/Papers/Paper_4/DAY-3/igem_parts.csv', index=False)
print(" Saved igem parts dataset to Drive!")

```

✓ Saved igem parts dataset to Drive!

!pip install GEOParse

```

Collecting GEOParse
  Downloading GEOParse-2.0.4-py3-none-any.whl.metadata (6.5 kB)
Requirement already satisfied: numpy>=1.7 in /usr/local/lib/python3.12/dist-packages (from GEOParse) (2.0.2)
Requirement already satisfied: pandas>=0.17 in /usr/local/lib/python3.12/dist-packages (from GEOParse) (2.2.2)
Requirement already satisfied: requests>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from GEOParse) (2.32.4)
Requirement already satisfied: tqdm>=4.31.1 in /usr/local/lib/python3.12/dist-packages (from GEOParse) (4.67.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.17->GEOParse) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.17->GEOParse) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.17->GEOParse) (2025.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.21.0->GEOParse) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.21.0->GEOParse) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.21.0->GEOParse) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.21.0->GEOParse) (2025.10.5)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=0.17->GEOParse) (1.17.0)
Downloading GEOParse-2.0.4-py3-none-any.whl (29 kB)
Installing collected packages: GEOParse
Successfully installed GEOParse-2.0.4

```

```

import GEOParse
import pandas as pd
from google.colab import drive

# 1 Mount Google Drive
drive.mount('/content/drive')

# 2 Set save path
save_path = '/content/drive/MyDrive/Papers/Paper_4/DAY-3/'

# 3 Download and parse GEO dataset
print("Downloading and parsing GEO dataset...")
gse = GEOParse.get_GEO("GSE19899", destdir="", how="full")

# 4 Display dataset information safely
print("\n== Dataset Metadata ==")
try:
    print("Title:", gse.metadata.get("title", ["N/A"])[0])
except Exception:
    print("Title info not found in metadata.")

print("\nNumber of samples:", len(gse.gsms))
print("Number of platforms:", len(gse.gpls))

# 5 Convert first sample to a DataFrame and preview
sample_id = list(gse.gsms.keys())[0]
sample = gse.gsms[sample_id]
print(f"\nFirst sample ID: {sample_id}")
df = sample.table

```

```
print("\n=== Sample Data Preview ===")
print(df.head())

# Save the sample data to CSV
output_file = save_path + "GSE19899_sample.csv"
df.to_csv(output_file, index=False)
print(f"\nSample data saved to: {output_file}")

20-Oct-2025 13:47:23 DEBUG utils - Directory . already exists. Skipping.
DEBUG:GEOparse:Directory . already exists. Skipping.
20-Oct-2025 13:47:23 INFO GEOparse - File already exist: using local version.
INFO:GEOparse:File already exist: using local version.
20-Oct-2025 13:47:23 INFO GEOparse - Parsing ./GSE19899_family.soft.gz:
INFO:GEOparse:Parsing ./GSE19899_family.soft.gz:
20-Oct-2025 13:47:23 DEBUG GEOparse - DATABASE: GeoMime
DEBUG:GEOparse:DATABASE: GeoMime
20-Oct-2025 13:47:23 DEBUG GEOparse - SERIES: GSE19899
DEBUG:GEOparse:SERIES: GSE19899
20-Oct-2025 13:47:23 DEBUG GEOparse - PLATFORM: GPL570
DEBUG:GEOparse:PLATFORM: GPL570
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Downloading and parsing GEO dataset...
/usr/local/lib/python3.12/dist-packages/GEOparse/GEOparse.py:401: DtypeWarning: Columns (2) have mixed types. Specify dtype option on import or set low_memory=False.
return read_csv(StringIO(data), index_col=None, sep="\t")
20-Oct-2025 13:47:25 DEBUG GEOparse - PLATFORM: GPL9052
DEBUG:GEOparse:PLATFORM: GPL9052
20-Oct-2025 13:47:25 DEBUG GEOparse - SAMPLE: GSM496089
DEBUG:GEOparse:SAMPLE: GSM496089
20-Oct-2025 13:47:25 DEBUG GEOparse - SAMPLE: GSM496090
DEBUG:GEOparse:SAMPLE: GSM496090
20-Oct-2025 13:47:26 DEBUG GEOparse - SAMPLE: GSM496091
20-Oct-2025 13:47:31 DEBUG GEOparse - SAMPLE: GSM497499
DEBUG:GEOparse:SAMPLE: GSM497499
20-Oct-2025 13:47:31 DEBUG GEOparse - SAMPLE: GSM497500
DEBUG:GEOparse:SAMPLE: GSM497500

=== Dataset Metadata ===
Title: Dissecting the unique role of the retinoblastoma tumor suppressor during cellular senescence

Number of samples: 50
Number of platforms: 2

First sample ID: GSM496089

=== Sample Data Preview ===
  ID_REF  VALUE
0  1007_s_at  5.784551
1   1053_at  5.996028
2   117_at   2.980048
3   121_at   4.075790
4  1255_g_at  2.267998

[✓] Sample data saved to: /content/drive/MyDrive/Papers/Paper_4/DAY-3/GSE19899_sample.csv

import pandas as pd

# Path to your DrugAge CSV
drugage_path = '/content/drive/MyDrive/Papers/Paper_4/DAY-3/drugage.csv'

# Load the dataset
drugage_df = pd.read_csv(drugage_path)

# Inspect first 5 rows
print("Shape:", drugage_df.shape)
drugage_df.head()

# Optional: Save a cleaned version (if needed)
drugage_df.to_csv('/content/drive/MyDrive/Papers/Paper_4/DAY-3/drugage_cleaned.csv', index=False)

Shape: (3423, 15)

# Check all column names in the dataset
import pandas as pd

drugage_path = '/content/drive/MyDrive/Papers/Paper_4/DAY-3/drugage.csv'
df = pd.read_csv(drugage_path)
print(df.columns.tolist())

rcent', 'avg_lifespan_significance', 'max_lifespan_change_percent', 'max_lifespan_significance', 'gender', 'weight_change_percent', 'weight_change_significance', 'ITP', 'pubmed_id']
```

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import roc_auc_score, average_precision_score, confusion_matrix

# Load dataset
drugage_path = '/content/drive/MyDrive/Papers/Paper_4/DAY-3/drugage.csv'
df = pd.read_csv(drugage_path)

# Create binary label
df['label'] = (df['avg_lifespan_change_percent'] > 0).astype(int)

# Helper function to extract numeric values from strings like '100 µM' or '5 days'
def extract_numeric(col):
    return col.astype(str).str.extract(r'([+-]?[d*\.\?d+])')[0].astype(float)

# Clean feature columns
df['dosage_num'] = extract_numeric(df['dosage'])
df['age_at_initiation_num'] = extract_numeric(df['age_at_initiation'])
df['treatment_duration_num'] = extract_numeric(df['treatment_duration'])

# Select numeric features
feature_cols = ['dosage_num', 'age_at_initiation_num', 'treatment_duration_num',
                'avg_lifespan_change_percent', 'max_lifespan_change_percent',
                'weight_change_percent']

X = df[feature_cols].fillna(0)
y = df['label']

```

```

# Split train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Baseline models
models = {
    'LogisticRegression': LogisticRegression(max_iter=1000),
    'GradientBoosting': GradientBoostingClassifier(),
    'SmallNN': MLPClassifier(hidden_layer_sizes=(10,), max_iter=500)
}

results = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:,1]
    results[name] = {
        'AUROC': roc_auc_score(y_test, y_prob),
        'PR': average_precision_score(y_test, y_prob),
        'ConfusionMatrix': confusion_matrix(y_test, y_pred)
    }

# Print results
for name, metrics in results.items():
    print(f"Model: {name}")
    print(f"AUROC: {metrics['AUROC']}")
    print(f"PR: {metrics['PR']}")
    print(f"Confusion Matrix:\n", metrics['ConfusionMatrix'])
    print("-----")

```

```

{
  Model: LogisticRegression
  AUROC: 0.9999900533142356
  PR: 0.999995520837066
  Confusion Matrix:
  [[213  0]
   [ 2 470]]
  -----
  Model: GradientBoosting
  AUROC: 0.9986124373358796
  PR: 0.999341209946802
  Confusion Matrix:
  [[213  0]
   [ 1 471]]
  -----
  Model: SmallNN
  AUROC: 0.9913961168138776
  PR: 0.992092625176497
  Confusion Matrix:
  [[196 17]
   [ 4 468]]
  -----
}

```

```

import pandas as pd
import requests
from time import sleep

# Load CellAge dataset
cellage_path = "/content/drive/MyDrive/Papers/Paper_4/DAY-3/cellage3.csv"
cellage_df = pd.read_csv(cellage_path)
print("Columns:", cellage_df.columns)
print("Number of genes:", len(cellage_df))

```

```

# 2 Use 'Gene symbol' column for motif scan
gene_list = cellage_df['gene symbol'].dropna().unique().tolist()
print(f"Unique genes for motif scan: {len(gene_list)}")

# 3 Fetch promoter sequences from Ensembl REST API
def fetch_promoter(gene_symbol, upstream=1000, downstream=100):
    """
    Fetch promoter sequence from Ensembl REST API
    upstream: bases upstream of TSS
    downstream: bases downstream of TSS
    """
    server = "https://rest.ensembl.org"
    ext = f"/sequence/id/{gene_symbol}?type=genomic;species=homo_sapiens;expand_5prime={upstream};expand_3prime={downstream}"
    headers = {"Content-type": "text/plain"}

    r = requests.get(server+ext, headers=headers)
    if r.status_code == 200:
        return r.text
    else:
        return None

# 4 Fetch promoters for first 50 genes (quick iteration)
promoter_dict = {}
for gene in gene_list[:50]:
    seq = fetch_promoter(gene)
    if seq:
        promoter_dict[gene] = seq
    sleep(0.1) # avoid hitting API too fast

print(f"Fetched promoters for {len(promoter_dict)} genes")

```

```

# 5 Save promoters to CSV for downstream motif scan
promoter_df = pd.DataFrame(list(promoter_dict.items()), columns=['Gene', 'Promoter sequence'])
save_path = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/cellage_promoters_v0.csv"
promoter_df.to_csv(save_path, index=False)
print("Promoters saved to:", save_path)

```

```

Columns: Index(['Entrez ID', 'Gene symbol', 'Gene name', 'Cancer Cell',
              'Type of senescence', 'Senescence Effect', 'Reference'],
              dtype='object')
Number of genes: 949
Unique genes for motif scan: 866
Fetched promoters for 0 genes
Promoters saved to: /content/drive/MyDrive/Papers/Paper 4/DAY-3/cellage_promoters_v0.csv

```

```

import pandas as pd
from Bio import motifs
from Bio.Seq import Seq
from collections import Counter

# 1 Load promoter sequences
promoter_path = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/cellage_promoters_v0.csv"
promoter_df = pd.read_csv(promoter_path)
print("Number of promoters:", len(promoter_df))

# 2 Extract sequences
sequences = [Seq(seq) for seq in promoter_df['Promoter sequence']]

# 3 Define function to scan for k-mers (length 6 for quick enrichment)
def get_kmer_counts(seqs, k=6):
    kmer_counter = Counter()
    for seq in seqs:
        seq_str = str(seq).upper()
        for i in range(len(seq_str)-k+1):
            kmer = seq_str[i:i+k]
            if 'N' not in kmer: # skip ambiguous bases
                kmer_counter[kmer] += 1
    return kmer_counter

# 4 Count k-mers
k = 6
kmer_counts = get_kmer_counts(sequences, k=k)
top_kmers = kmer_counts.most_common(20) # top 20 motifs
print(f"Top {len(top_kmers)} candidate motifs (length {k}):")
for motif, count in top_kmers:
    print(motif, count)

```

```
# Save motif shortlist
motif_df = pd.DataFrame(top_kmers, columns=['Motif', 'count'])
save_path = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/candidate_motifs_v0.csv"
motif_df.to_csv(save_path, index=False)
print("Motif shortlist saved to:", save_path)
```

Number of promoters: 0
Top 0 candidate motifs (length 6):
Motif shortlist saved to: /content/drive/MyDrive/Papers/Paper 4/DAY-3/candidate_motifs_v0.csv

```
# Step 3
from pyensembl import EnsemblRelease
from Bio.Seq import Seq
import pandas as pd

# Load Cellage genes
cellage_path = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/cellage3.csv"
cellage_df = pd.read_csv(cellage_path)
genes = cellage_df['Gene symbol'].tolist()
print(f"Number of genes: {len(genes)}")

# Initialize Ensembl human release
ensembl = EnsemblRelease(108) # latest stable release

promoter_records = []

for gene_name in genes:
    try:
        gene = ensembl.gene_by_name(gene_name)
```

```
    for gene_name in genes:
        try:
            gene = ensembl.gene_by_name(gene_name)
            transcripts = gene.transcripts
            if transcripts:
                t = transcripts[0] # take first transcript
                if t.strand == 1:
                    start = max(0, t.start - 1000)
                    end = t.start - 1
                else:
                    start = t.end + 1
                    end = t.end + 1000
                seq = ensembl.genome_sequence(contig=t.contig, start=start, end=end, strand=t.strand)
                promoter_records.append({'Gene_symbol': gene_name, 'Promoter_sequence': seq})
            except Exception as e:
                print(f"Skipping {gene_name}: {e}")

        promoter_df = pd.DataFrame(promoter_records)
        print(f"Number of promoters fetched: {len(promoter_df)}")

        # Save promoter sequences CSV
        promoter_save_path = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/promoters_v0.csv"
        promoter_df.to_csv(promoter_save_path, index=False)
        print("Promoter sequences saved to:", promoter_save_path)
```

Number of genes: 949
Skipping AAK1: 'EnsemblRelease' object has no attribute 'gene_by_name'
Skipping ABCB1: 'EnsemblRelease' object has no attribute 'gene_by_name'
Skipping ABCG6: 'EnsemblRelease' object has no attribute 'gene_by_name'
Skipping ABT3: 'EnsemblRelease' object has no attribute 'gene_by_name'

```
!ls "/content/drive/MyDrive/Papers/Paper 4/DAY-3/"
```

candidate_motifs_v0.csv	cellage_promoters_v0.csv	GSE19899_sample.csv
cellage3.csv	drugage_cleaned.csv	igem_parts.csv
cellage3.tsv	drugage.csv	promoters_v0.csv

```
import pandas as pd

# Load dataset
data_file = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/cellage3.csv"
df = pd.read_csv(data_file)

# Quick check
print("Columns in dataset:", df.columns)
print("Shape of dataset:", df.shape)

# Take a slice for iteration (e.g., first 1000 rows)
df_slice = df.head(1000)

# Set label column based on your dataset
label_column = 'Senescence Effect' # Correct label

# Features: drop the label and reference columns
X = df_slice.drop(columns=[label_column, 'Reference'])
y = df_slice[label_column]

print("Shape of features (X):", X.shape)
print("Shape of labels (y):", y.shape)
print("First few labels:")
```

```

print("Shape of features (X):", X.shape)
print("Shape of labels (y):", y.shape)
print("First few labels:")
print(y.head())

```

⌕ Rectangular Snip

```

Columns in dataset: Index(['Entrez ID', 'Gene symbol', 'Gene name', 'Cancer Cell',
                          'Type of senescence', 'Senescence Effect', 'Reference'],
                          dtype='object')
Shape of dataset: (949, 7)
Shape of features (X): (949, 5)
Shape of labels (y): (949,)
First few labels:
0    Induces
1    Induces
2    Inhibits
3    Induces
4    Induces
Name: Senescence Effect, dtype: object

```

```

import pandas as pd

# Load promoter sequences
promoter_file = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/cellage_promoters_v0.csv"
promoters_df = pd.read_csv(promoter_file)

# Show first 5 rows and columns
print("Columns in promoter dataset:")
print(promoters_df.columns)
print("\nFirst 5 rows:")
print(promoters_df.head())

```

```

Columns in promoter dataset:
Index(['Gene', 'Promoter_sequence'], dtype='object')

First 5 rows:
Empty DataFrame
Columns: [Gene, Promoter_sequence]
Index: []

```

⌕ Rectangular Snip

```

import pandas as pd
import requests
import time

# Load gene list
gene_file = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/cellage3.csv"
df_genes = pd.read_csv(gene_file)
gene_list = df_genes['Gene symbol'].dropna().unique().tolist()
print(f"Number of candidate genes: {len(gene_list)}")

# Function to convert symbol -> Ensembl ID
def get_ensembl_id(symbol, species="human"):
    server = "https://rest.ensembl.org"
    url = f"{server}/lookup/symbol/{species}/{symbol}?content-type=application/json"
    try:
        response = requests.get(url, headers={"Content-Type": "application/json"})
        if response.ok:
            return response.json().get("id", None)
        else:
            print(f"Skipping {symbol}: {response.status_code}")
            return None
    except Exception as e:

```

```

        print(f"Error with {symbol}: {e}")
        return None

# Function to fetch promoter sequence using Ensembl ID
def fetch_promoter_sequence(ensembl_id, upstream=1000, downstream=100):
    server = "https://rest.ensembl.org"
    url = f"{server}/sequence/id/{ensembl_id}?content-type=application/json;expand_5prime={upstream};expand_3prime={downstream}"
    try:
        response = requests.get(url, headers={"Content-Type": "application/json"})
        if response.ok:
            return response.json().get("seq", None)
        else:
            print(f"Skipping {ensembl_id}: {response.status_code}")
            return None
    except Exception as e:
        print(f"Error fetching {ensembl_id}: {e}")
        return None

# Fetch promoters
promoter_records = []

for gene in gene_list:
    ensembl_id = get_ensembl_id(gene)
    if ensembl_id:
        seq = fetch_promoter_sequence(ensembl_id)
        if seq:
            promoter_records.append({"Gene": gene, "Promoter_sequence": seq})
        time.sleep(0.1) # polite delay

promoters_df = pd.DataFrame(promoter_records)
print(f"Promoter sequences fetched: {len(promoters_df)}")

```

⌕ Rectangular Snip

```

# Save
output_file = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/ensembl_promoters_v0.csv"
promoters_df.to_csv(output_file, index=False)
print(f"Promoter sequences saved to: {output_file}")

# Show first few
promoters_df.head()

```

Number of candidate genes: 866
 Skipping PHB: 400
 Promoter sequences fetched: 865
 Promoter sequences saved to: /content/drive/MyDrive/Papers/Paper 4/DAY-3/ensembl_promoters_v0.csv

	Gene	Promoter_sequence
0	AAK1	TTTATTTGAAGTCATATCATATACGTAAGTAAATAACAGAATAT...
1	ABCB1	TAAATCCCTCACTAACCTACCCCTGCCCTCACTAAACTTAATAATA...
2	ABCC6	ATATCACAGTGTGTACTCCCACTGTGATATTAGGAGTAAATCTCC...
3	ABI3	CACCAAGCTGTGCCACTTAGTACTTGGGGCGTGGCCACCTCTAATG...
4	ABI3BP	TTGATACTGTCATCTTTATTTAGAATCTAAATGTAGTAATTAAG...

```

import os

base_path = "/content/drive/MyDrive/Papers/Paper 4/DAY-3"
files = os.listdir(base_path)
print("Files in DAY-3 folder:\n")
for f in files:
    print(f)

```

Files in DAY-3 folder:

cellage3.tsv
 cellage3.csv
 igem_parts.csv
 GSE19899_sample.csv
 drugage.csv
 drugage_cleaned.csv
 cellage_promoters_v0.csv
 candidate_motifs_v0.csv
 promoters_v0.csv
 ensembl_promoters_v0.csv

```

import pandas as pd
from collections import Counter
import re

# Load promoter sequences (confirmed existing file)
promoter_file = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/ensembl_promoters_v0.csv"
df = pd.read_csv(promoter_file)

print("Columns in promoter dataset:", df.columns)
print("Number of promoter entries:", len(df))

# Ensure we have the right column
seq_col = [c for c in df.columns if 'sequence' in c.lower()]
if not seq_col:
    raise ValueError("No column containing promoter sequences found.")
seq_col = seq_col[0]

# Drop missing values
df = df.dropna(subset=[seq_col])

# Motif discovery (simple 6-mer enrichment)
def find_motifs(seq, k=6):
    return [seq[i:i+k] for i in range(len(seq)-k+1)]

motif_counts = Counter()
for seq in df[seq_col].astype(str):
    seq = seq.upper()
    if re.match(r'^[ACGT]+$', seq):
        motif_counts.update(find_motifs(seq))

```



```
# Save output
output_file = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/motif_enrichment_v0.csv"
motifs_df.to_csv(output_file, index=False)

print(f"\nTop {len(motifs_df)} candidate motifs saved to: {output_file}")
print(motifs_df)
```

Columns in promoter dataset: Index(['Gene', 'Promoter_sequence'], dtype='object')
Number of promoter entries: 865

Top 20 candidate motifs saved to: /content/drive/MyDrive/Papers/Paper 4/DAY-3/motif_enrichment_v0.csv

	Motif	Count
443	TTTTTT	305574
375	AAAAAA	207063
399	ATTTTT	120927
2	TATTTT	102426
444	TTTTTA	100207
414	AAAAAT	91188
1	TTATTT	88125
1193	TTTCTT	86392
411	TTTTAA	86189
0	TTTATT	85341
573	TTTCTT	84036
400	TTTTTG	83988
412	TTTAAA	79567
574	TTCTTT	78897
398	AATTTT	78166
1408	TTTTGT	77546
110	AAAATA	77499
1694	TGTTTT	76871
445	TTTTAT	76184
576	CTTTTT	74035

```
import pandas as pd
import numpy as np
import os

# === Step 1: Load promoter and motif enrichment data ===
promoter_file = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/ensembl_promoters_v0.csv"
motif_file = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/motif_enrichment_v0.csv"

promoters_df = pd.read_csv(promoter_file)
motifs_df = pd.read_csv(motif_file)

print("Promoter columns:", promoters_df.columns.tolist())
print("Motif columns:", motifs_df.columns.tolist())

# === Step 2: Define "leaky" motifs ===
leaky_motifs = ['TTTTTT', 'AAAAAA', 'ATTTTT', 'TTTTTA', 'TTTATA', 'TTTTAT', 'TTTTGT']

# === Step 3: Safety metric calculations ===
def compute_safety_metrics(promoter_seq):
    if pd.isna(promoter_seq):
        return pd.Series({'leakiness_score': np.nan, 'specificity_score': np.nan, 'robustness_score': np.nan})

    motif_counts = {m: promoter_seq.count(m) for m in leaky_motifs}
    total_leaky = sum(motif_counts.values())
    total_len = len(promoter_seq)

    leakiness_score = total_leaky / total_len if total_len > 0 else 0
    specificity_score = len([m for m, c in motif_counts.items() if c == 0]) / len(leaky_motifs)
    robustness_score = (1 - leakiness_score) * specificity_score

    return pd.Series({'leakiness_score': leakiness_score,
                      'specificity_score': specificity_score,
                      'robustness_score': robustness_score})
```

```
return pd.Series([
    'leakiness_score': leakiness_score,
    'specificity_score': specificity_score,
    'robustness_score': robustness_score
])

# Apply to dataset
print("Computing safety metrics...")
metrics_df = promoters_df.copy()
metrics_df = pd.concat([metrics_df, promoters_df['Promoter_sequence'].apply(compute_safety_metrics)], axis=1)

# === Step 4: Assign safety levels ===
def assign_safety_category(row):
    if row['leakiness_score'] < 0.002 and row['robustness_score'] > 0.7:
        return 'SAFE'
    elif row['leakiness_score'] < 0.005 and row['robustness_score'] > 0.4:
        return 'MODERATE'
    else:
        return 'LEAKY'

metrics_df['safety_category'] = metrics_df.apply(assign_safety_category, axis=1)

# === Step 5: Save results ===
output_file = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/promoter_safety_scores_v1.csv"
metrics_df.to_csv(output_file, index=False)

print(f"Safety metrics calculated and saved to: {output_file}")
print(metrics_df[['Gene', 'leakiness_score', 'specificity_score', 'robustness_score', 'safety_category']].head(10))
```

```

Promoter columns: ['Gene', 'Promoter_sequence']
Motif columns: ['Motif', 'Count']
computing safety metrics...
Safety metrics calculated and saved to: /content/drive/MyDrive/Papers/Paper 4/DAY-3/promoter_safety_scores_v1.csv

```

	Gene	leakiness_score	specificity_score	robustness_score
0	AAK1	0.010273	0.0	0.0
1	ABCB1	0.010380	0.0	0.0
2	ABCC6	0.005840	0.0	0.0
3	ABI3	0.004096	0.0	0.0
4	ABI3BP	0.011203	0.0	0.0
5	ACER2	0.010046	0.0	0.0
6	ACKR1	0.002308	0.0	0.0
7	ACLY	0.008744	0.0	0.0
8	ADCK5	0.004802	0.0	0.0
9	AGO2	0.007429	0.0	0.0

	safety_category
0	LEAKY
1	LEAKY
2	LEAKY
3	LEAKY
4	LEAKY
5	LEAKY
6	LEAKY
7	LEAKY
8	LEAKY
9	LEAKY

```

import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load the gene dataset
gene_file = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/cellage3.csv"
df = pd.read_csv(gene_file)

print("Columns in dataset:", df.columns)

# Drop rows with missing label
df = df.dropna(subset=['Senescence Effect'])

# Encode the label as binary (example: Pro-senescent=1, Anti-senescent=0)
label_encoder = LabelEncoder()
df['Senescence_Label'] = label_encoder.fit_transform(df['Senescence Effect'])

# Features: let's take 'Cancer Cell' and 'Type of senescence'
feature_cols = ['Cancer Cell', 'Type of senescence']
X = pd.get_dummies(df[feature_cols], drop_first=True)
y = df['Senescence_Label']

print("Feature matrix shape:", X.shape)
print("Label vector shape:", y.shape)

```

```

Columns in dataset: Index(['Entrez ID', 'Gene symbol', 'Gene name', 'Cancer Cell',
                          'Type of senescence', 'Senescence Effect', 'Reference'],
                          dtype='object')
Feature matrix shape: (949, 4)
Label vector shape: (949,)

```

```

from sklearn.preprocessing import label_binarize

# Binarize the labels for multi-class metrics
classes = y.unique()
y_test_bin = label_binarize(y_test, classes=classes)
y_prob_bin = lr_model.predict_proba(X_test)

# Compute AUROC (one-vs-rest)
auroc = roc_auc_score(y_test_bin, y_prob_bin, average='macro', multi_class='ovr')
aupr = average_precision_score(y_test_bin, y_prob_bin, average='macro')

# Confusion matrix (still works as usual)
cm = confusion_matrix(y_test, y_pred)

print(f"Logistic Regression AUROC (macro, OVR): {auroc:.4f}")
print(f"Logistic Regression AUPR (macro): {aupr:.4f}")
print("Confusion Matrix:")
print(cm)

```

```

Logistic Regression AUROC (macro, OVR): 0.6209
Logistic Regression AUPR (macro): 0.4477
Confusion Matrix:
[[29 55  0]
 [12 90  0]
 [ 0  4  0]]

```

<pre> from sklearn.ensemble import GradientBoostingClassifier # Initialize GBM gbm_model = GradientBoostingClassifier(n_estimators=200, learning_rate=0.1, max_depth=3, random_state=42) # Train gbm_model.fit(X_train, y_train) # Predict y_pred_gbm = gbm_model.predict(X_test) y_prob_gbm = gbm_model.predict_proba(X_test) # Binarize labels for multi-class metrics y_test_bin = label_binarize(y_test, classes=y.unique()) # Compute metrics auroc_gbm = roc_auc_score(y_test_bin, y_prob_gbm, average='macro', multi_class='ovr') aupr_gbm = average_precision_score(y_test_bin, y_prob_gbm, average='macro') cm_gbm = confusion_matrix(y_test, y_pred_gbm) print(f"GBM AUROC (macro, OVR): {auroc_gbm:.4f}") print(f"GBM AUPR (macro): {aupr_gbm:.4f}") print("GBM Confusion Matrix:") print(cm_gbm) </pre>	
<pre> GBM AUROC (macro, OVR): 0.6222 GBM AUPR (macro): 0.4377 GBM Confusion Matrix: [[40 44 0] [23 79 0] [0 4 0]] </pre>	
<pre> import torch import torch.nn as nn from torch.utils.data import TensorDataset, DataLoader # Convert data to tensors X_train_tensor = torch.tensor(X_train.values, dtype=torch.float32) y_train_tensor = torch.tensor(y_train.values, dtype=torch.long) X_test_tensor = torch.tensor(X_test.values, dtype=torch.float32) y_test_tensor = torch.tensor(y_test.values, dtype=torch.long) # Dataset & DataLoader train_dataset = TensorDataset(X_train_tensor, y_train_tensor) train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True) # Simple feedforward NN class SmallNN(nn.Module): def __init__(self, input_dim, num_classes): super(SmallNN, self).__init__() self.fc1 = nn.Linear(input_dim, 32) self.relu = nn.ReLU() self.fc2 = nn.Linear(32, num_classes) def forward(self, x): x = self.relu(self.fc1(x)) x = self.fc2(x) return x input_dim = X_train.shape[1] num_classes = len(y.unique()) model = SmallNN(input_dim, num_classes) </pre>	
<pre> # Loss and optimizer criterion = nn.CrossEntropyLoss() optimizer = torch.optim.Adam(model.parameters(), lr=0.001) # Train loop for epoch in range(50): for xb, yb in train_loader: optimizer.zero_grad() outputs = model(xb) loss = criterion(outputs, yb) loss.backward() optimizer.step() # Predictions with torch.no_grad(): outputs_test = model(X_test_tensor) y_prob_nn = torch.softmax(outputs_test, dim=1).numpy() y_pred_nn = y_prob_nn.argmax(axis=1) # Compute metrics y_test_bin = label_binarize(y_test, classes=y.unique()) auroc_nn = roc_auc_score(y_test_bin, y_prob_nn, average='macro', multi_class='ovr') aupr_nn = average_precision_score(y_test_bin, y_prob_nn, average='macro') cm_nn = confusion_matrix(y_test, y_pred_nn) print(f"NN AUROC (macro, OVR): {auroc_nn:.4f}") print(f"NN AUPR (macro): {aupr_nn:.4f}") print("NN Confusion Matrix:") print(cm_nn) </pre>	

```

NN AUROC (macro, OVR): 0.6269
NN AUPR (macro): 0.4466
NN Confusion Matrix:
[[48 36  0]
 [27 75  0]
 [ 0  4  0]]

```

```

# Step 3 Baseline: Small Neural Network (NN)
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import roc_auc_score, average_precision_score, confusion_matrix
import numpy as np

# --- Prepare data ---
# X_train, X_test, y_train, y_test are assumed ready from previous steps

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Encode labels to integers
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)
num_classes = len(le.classes_)

```

```

# Convert to PyTorch tensors
X_train_tensor = torch.tensor(X_train_scaled, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train_enc, dtype=torch.long)
X_test_tensor = torch.tensor(X_test_scaled, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test_enc, dtype=torch.long)

# Create DataLoader
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

# --- Define model ---
class SmallNN(nn.Module):
    def __init__(self, input_dim, hidden_dim=32, output_dim=num_classes):
        super(SmallNN, self).__init__()
        self.layers = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, output_dim)
        )

    def forward(self, x):
        return self.layers(x)

model = SmallNN(X_train_tensor.shape[1])
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# --- Train model ---
epochs = 50
for epoch in range(epochs):
    model.train()

```

```

        if (epoch+1) % 10 == 0:
            print(f"Epoch {epoch+1}/{epochs}, Loss: {loss.item():.4f}")

# --- Evaluate model ---
model.eval()
with torch.no_grad():
    outputs = model(X_test_tensor)
    y_prob = nn.Softmax(dim=1)(outputs).numpy()
    y_pred = np.argmax(y_prob, axis=1)

# Compute metrics
auroc = roc_auc_score(y_test_enc, y_prob, multi_class='ovr', average='macro')
aupr = average_precision_score(y_test_enc, y_prob, average='macro')
cm = confusion_matrix(y_test_enc, y_pred)

print(f"Small NN AUROC (macro, OVR): {auroc:.4f}")
print(f"Small NN AUPR (macro): {aupr:.4f}")
print("Small NN Confusion Matrix:")
print(cm)

```

```

Epoch 10/50, Loss: 0.9644
Epoch 20/50, Loss: 0.7816
Epoch 30/50, Loss: 0.7252
Epoch 40/50, Loss: 0.6640
Epoch 50/50, Loss: 0.9563
Small NN AUROC (macro, OVR): 0.6012
Small NN AUPR (macro): 0.4351
Small NN Confusion Matrix:
[[40 44  0]
 [23 79  0]
 [ 0  4  0]]

```

```
import pandas as pd
import numpy as np

# -----
# 1. Baseline metrics
# -----
# Replace these arrays with the predictions from your previous runs
# If you still have the results in memory:
# For example purposes, I will create placeholders

baseline_metrics = []

# Logistic Regression (fill with actual numbers from your previous output)
baseline_metrics.append({
    'Model': 'Logistic Regression',
    'AUROC_macro_OVR': 0.6209,
    'AUPR_macro': 0.4477,
    'Confusion_Matrix': [[29,55,0],[12,90,0],[0,4,0]]
})

# Gradient Boosting (GBM)
baseline_metrics.append({
    'Model': 'Gradient Boosting (GBM)',
    'AUROC_macro_OVR': 0.6222,
    'AUPR_macro': 0.4377,
    'Confusion_Matrix': [[40,44,0],[23,79,0],[0,4,0]]
})

# Small Neural Network
baseline_metrics.append({
    'Model': 'Small NN',
    'AUROC_macro_OVR': 0.6012,
    'AUPR_macro': 0.4351,
    'Confusion_Matrix': [[40,44,0],[23,79,0],[0,4,0]]
})

baseline_df = pd.DataFrame(baseline_metrics)
baseline_df.to_csv("/content/drive/MyDrive/Papers/Paper 4/DAY-3/final_baseline_metrics.csv", index=False)
print("✅ Baseline metrics saved to final_baseline_metrics.csv")

# -----
# 2. Motif shortlist
# -----
motif_file = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/motif_enrichment_v0.csv"
motif_df = pd.read_csv(motif_file)
motif_df.to_csv("/content/drive/MyDrive/Papers/Paper 4/DAY-3/final_motif_shortlist.csv", index=False)
print("✅ Motif shortlist saved to final_motif_shortlist.csv")

# -----
# 3. Promoter safety scores
# -----
safety_file = "/content/drive/MyDrive/Papers/Paper 4/DAY-3/promoter_safety_scores_v1.csv"
safety_df = pd.read_csv(safety_file)
safety_df.to_csv("/content/drive/MyDrive/Papers/Paper 4/DAY-3/final_promoter_safety_scores.csv", index=False)
print("✅ Promoter safety scores saved to final_promoter_safety_scores.csv")

✅ Baseline metrics saved to final_baseline_metrics.csv
✅ Motif shortlist saved to final_motif_shortlist.csv
✅ Promoter safety scores saved to final_promoter_safety_scores.csv
```